

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Фізико-технічний інститут

КРИПТОГРАФІЯ

Виконав:

Студент гр. ФБ-31 Моїсеєнко Д.Ю.

Київ-2026

Комп'ютерний практикум №4. Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Мета роботи: Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів

4 варіант

Порядок виконання роботи:

- 1. Написати функцію пошуку випадкового простого числа з заданого інтервалу або заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на простоту. В якості датчика випадкових чисел використовуйте вбудований генератор псевдовипадкових чисел вашої мови програмування. В якості тесту перевірки на простоту рекомендовано використовувати тест Міллера-Рабіна із попередніми пробними діленнями. Тести необхідно реалізовувати власноруч, використання готових реалізацій тестів не дозволяється.*
- 2. За допомогою цієї функції згенерувати дві пари простих чисел p, q і $1 \leq p, q$ довжини щонайменше 256 біт. При цьому пари чисел беруться так, щоб $p \cdot q \leq 2^{256}$; p і q – прості числа для побудови ключів абонента A , $1 \leq p$ і $q \leq 2^{256}$ – абонента B .*
- 3. Написати функцію генерації ключових пар для RSA. Після генерування функція повинна повертати та/або зберігати секретний ключ (d, p, q) та відкритий ключ (n, e) . За допомогою цієї функції побудувати схеми RSA для абонентів A і B – тобто, створити та зберегти для подальшого використання відкриті ключі (e, n) , (e, n) і $n \leq 2^{256}$ та секретні d і $d \leq 2^{256}$.*
- 4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів A і B . Кожна з операцій (шифрування, розшифрування, створення цифрового підпису, перевірка цифрового підпису) повинна бути реалізована окремою процедурою, на вхід до якої повинні подаватись лише ті ключові дані, які необхідні для її виконання. За допомогою датчика випадкових чисел вибрати відкрите повідомлення M і знайти криптограму для абонентів A і B , перевірити правильність розшифрування. Скласти для A і B повідомлення з цифровим підписом і перевірити його.*

5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA. Протоколи роботи кожного учасника (відправника та приймаючого) повинні бути реалізовані у вигляді окремих процедур, на вхід до яких повинні подаватись лише ті ключові дані, які необхідні для виконання. Перевірити роботу програм для випадково обраного ключа $0 < k < n$.

Кожна з наведених операцій повинна бути реалізована у вигляді окремої процедури, інтерфейс якої повинен приймати лише ті дані, які необхідні для її роботи; наприклад, функція `Encrypt()`, яка шифрує повідомлення для абонента, повинна приймати на вхід повідомлення та відкритий ключ адресата (і тільки його), повертаючи в якості результату шифротекст. Відповідно, програмний код повинен містити сім високорівневих процедур: `GenerateKeyPair()`, `Encrypt()`, `Decrypt()`, `Sign()`, `Verify()`, `SendKey()`, `ReceiveKey()`.

Кожну операцію рекомендується перевіряти шляхом взаємодії із тестовим середовищем, розташованим за адресою

<http://asymcryptwebservice.appspot.com/?section=rsa>.

Наприклад, для перевірки коректності операції шифрування необхідно а) зашифрувати власною реалізацією повідомлення для серверу та розшифрувати його на сервері, б) зашифрувати на сервері повідомлення для вашої реалізації та розшифрувати його локально.

Хід роботи:

1. Опис роботи та труднощі:

Використання перевірки на простоту тобто тест пробних ділень на малі прості числа і імовірнісний тест Міллера-Рабіна. Також піднесення степеня було використано схему Горнера.

2. Побудова криптосистеми RSA:

Абонент А:

p, q:

```
Генерація ключів для абонентів А та В
Абонент А:
p=100275095552114348978147934851688157735952692564612362675493458140889072505607
q=79239886314642147799883748899054613178026193498800126693820468583995628301497
```

n, e, d:

```
n=794578717173941950734848735221106086011491912852301130832277741348265794494914942204242776316360515760426003699124000
7612469334861658889429126179418993679
e=65537
d=61231676048877373514065032765623046323822994136024174306931276211190864476107107716536571830723770034468165151446275
7752085153478037028261490074132891265
```

Абонент В:

p1, q1:

```
Абонент В:
p=86813939107372821387410960037440180745264388484179664288852627577598630739371
q=112314235413168609687384403603289145863461728767221111309053527716917801767837
```

n1, e1, d1:

```
n=975044119404995581639717862203642862963048809935886819329984713250033877809812764817228852233980060697250563053589328
8510808546117927523098461274197410527
e=65537
d=571812040301075731033475995482472645911802767308175137924052557624728047663786540027151402380710084140829018131860846
3479164064941744304707168029783729713
```

3. Результати чисельних операцій:

```
Шифрування та Цифровий підпис
Відкрите повідомлення М: 13479251998880046287709055591232511520570867862759073960925893732222890216732255879954842919811
04228242281036536504032464767063195736844207825231624588205
Шифртекст (С): 14612959253139704196208953142728156742621844878260898362285258681094434797141199339702237610599763231111
6063743443015948225632221221461480470927645807161
Розшифроване повідомлення: 134792519988800462877090555912325115205708678627590739609258937322228902167322558799548429198
1104228242281036536504032464767063195736844207825231624588205
Перевірка розшифрування: Успішно
Цифровий підпис (S): 402408861136542525857946095431581492078365765709050998434019038304338998185269109668159638279318262
3920491365165175081860716807337949957424187659130812151
Перевірка підпису: Вірний
```

4. Опис кроків протоколу конфіденційного розсилання ключів:

4.1: Обрали секретний ключ k і обчислили зашифрований ключ $k_1 = k^{e_1} \bmod n_1$ та зашифрований підпис $S_1 = (k^d \bmod n) e_1 \bmod n_1$

```
Протокол конфіденційного розсилання ключів
Секретний ключ k для передачі: 19330056612408790423797108886665885775857054509317857822796960730323151069787520322103148
62937620358975854506829295546379180410539972733626387641668752873
Характеристики передачі:
k1 (шифр. ключ)=6470832961342640472595631081233591424309455462340400867040739261912297019058111690371607596988717120028
11042669244029409856699944268806712316277224286758
S1 (шифр. підпис)=43599962872804472802526972293164353720000566093278910225950794695296867392585430042863251926732824668
271230969045246015852068480991882558100243680288677
Отриманий ключ k: 193300566124087904237971088866658857758570545093178578227969607303231510697875203221031486293762035897
5854506829295546379180410539972733626387641668752873
Автентифікація відправника: Підтверджено
```

4.2: В результаті виконання розшифрування $k = k_1^{d_1} \bmod n_1$ та перевірка підпису $k = S^e \bmod n$. Автентифікація підтверджена.

5. Перевірка тестового середовища:

<http://asymcryptwebservice.appspot.com/?section=rsa>

5.1:

RSA Testing Environment

Server Key

Encryption

Decryption

Signature

Verification

Send Key

Receive Key

Get server key

✖ Clear

Key size

256

Get key

Modulus

BB1FAAF1B85C446D49712246C97546CCED79D743558A7DFC0ACE34AC9D09C397

Public exponent

10001

```
PS D:\3 курс 5 семестр 2025(академічна різниця)\Криптографія\Криптографія загал  
1_ср4> python lab4.py  
Шифртекст:  
35788243894364318005074569375362726406266230197952842674988068675913804053264
```

5.2:

RSA Testing Environment

Server Key

Encryption

Decryption

Signature

Verification

Send Key

Receive Key

Encryption

✖ Clear

Modulus

BB1FAAF1B85C446D49712246C97546CCED79D743558A7DFC0ACE34AC9D09C397

Public exponent

10001

Message

1234

Bytes

Encrypt

Ciphertext

578DC3A64C0A6733ADF83C6738B45D27C7880FD18B61859A6357F8BE1E4EE7B4

5.3:

RSA Testing Environment

Server Key

Encryption

Decryption

Signature

Verification

Send Key

Receive Key

Decryption

Clear

Ciphertext

578DC3A64C0A6733ADF83C6738B45D27C7880FD18B61859A6357F8BE1E4EE7

Bytes

Decrypt

Message

1234

5.4:

Server Key

Encryption

Decryption

Signature

Verification

Send Key

Receive Key

Sign

Clear

Message

1234

Bytes

Sign

Signature

2F58554E50A91123CF17BEDA6FCA31F3068A3E9A1E08ED32F02A98BB62EA6546

5.5:

Server Key

Encryption

Decryption

Signature

Verification

Send Key

Receive Key

Verify

Clear

Message

1234

Bytes

Signature

2F58554E50A91123CF17BEDA6FCA31F3068A3E9A1E08ED32F02A98BB62EA6546

Modulus

BB1FAAF1B85C446D49712246C97546CCED79D743558A7DFC0ACE34AC9D09C397

Public exponent

10001

Verify

Verification

true

Висновки:

Під час виконання комп'ютерного практикуму було реалізовано асиметричну криптосистему RSA. Імовірнісні тести простоти є ефективними для генерації ключів великої довжини. Також сумісність реалізації перевірена за допомогою тестового середовища RSA.